My first 90 days as Lexroom's first Analytics Engineer

Mattia Franzè, October 2026

Point of view. The first analytics hire is not paid to write SQL; they are paid to make a handful of numbers trustworthy enough that Product, CS, Sales and Engineering stop arguing about them. At Lexroom today, with events landing from Pub/Sub into BigQuery, no transformation layer and four teams reading them their own way, that means: start where decisions are already being made on shaky numbers, build the smallest thing that fixes one of them, and let every next step be pulled by a question someone actually asked. I come from a large, well-staffed warehouse build; the discipline this role needs in the first 90 days is the opposite, and I know it.

1. Priorities: 30 / 60 / 90 days

Days 1 to 30: learn how data is born, ship one trusted number.

- Map how events are produced and land: which services emit which topics, how schemas change, what happens on retries. The case data shows what to expect: duplicates, orphan keys, bad timestamps.
- First week in production: dbt Core on a scheduled job against BigQuery, whose only job at first is source freshness and daily row counts on the landing tables. Half a day of setup, and the first check that catches a real problem.
- One question to each team: "which number do you not trust today?" That list is the roadmap. Fifteen minutes a week with one named point of contact per team keeps it alive: not a committee, a channel for "this number looks wrong".
- First domain: AMA quality, because Product, CS and Leadership all read it and already compute it differently. One model holding the definitions, the weekly and per-account tables on top, docs a PM can read, tests on every pull request.

Days 31 to 60: second and third domains, pulled by questions.

- Likely candidates: workspace usage and activation (CS and Sales need it from the same table; it also needs plan and seat data from billing or the CRM, a small ingestion of its own) and module adoption per plan and country (Product). I will not know until day 30, and that is the point.
- A definitions page next to the models, surfaced where people already look, with each table linked to the question it answers.

Days 61 to 90: ownership, self-service, baseline SLAs.

- Every domain gets a business owner (a person, not a team) and a technical owner (me), and one engineer who can run and extend the project without me.
- A thin self-service layer on top of the tables, in whatever BI tool already exists, only for definitions that have survived 30 days of use.
- SLAs committed only after 30 days of measurement: for example "workspace table refreshed by 08:00 CET, alert if more than two hours late".
- A review with the CTO: what gets built next, and what gets stopped.

2. Data quality framework

Tests that would catch a real problem, in three tiers, and nothing else:

- **Sources:** freshness, row-count anomalies, known upstream issues as `warn` so they are visible on every run without blocking it.
- **Keys and grain:** uniqueness on the declared grain, relationships on the joins that feed a decision.
- **Reconciliation:** singular tests that prove the outputs add up to the inputs (every event counted once, rates between 0 and 1, breakdowns summing to totals). These catch fan-out and silent filters, the two bugs that quietly move every KPI.

Without slowing anyone down:

- Tests run in CI on every pull request, in a per-PR BigQuery dev dataset, only on the models that changed and their children. A failing test blocks the merge; a warning does not.
- Severity is a decision: `error` only when a wrong number would reach someone who acts on it.
- Boilerplate (YAML, basic tests, docs stubs) is generated with AI tooling, so time goes to definitions and edge cases.
- **Ownership** is a lightweight contract with Engineering: for the few events that feed a published table, a schema check in their CI (not a human review) fails fast and names what breaks downstream; everything else they change freely. In return they stop hearing about broken numbers from a stakeholder first.

- **SLAs** come last and come from data: a baseline first, then one per table, owned jointly with the engineer who produces the source.

3. Trust and self-service, team by team

- **Product** wants trends and reasons: module by week, with the "why" behind a thumbs down. Trust is earned when the table answers their question in one short query and the definition is written where they read it. Self-service: the weekly module table in the BI tool, with the low-sample flag visible so nobody acts on 12 ratings.
- **Customer Success** wants a list of accounts to call, with a reason each. Trust is earned by not crying wolf: usage measured against the platform trend, "not enough data" kept separate from "healthy", a drop confirmed in two windows before it tops the list. Self-service: one row per workspace, refreshed daily.
- **Sales** wants usage and adoption per workspace and plan for renewals and upsell, and will compare them with what CS sees. Trust is earned when both teams read the same row. Self-service: the same workspace table, with plan and seats, surfaced in the CRM if that is where they live.
- **Engineering** is both my main source and a consumer: delivery metrics are already on the list. Trust is earned by reporting problems with evidence (the query, the rows, the impact), never with "the data looks off", and by never asking for ad hoc extracts. What they get back: the schema check above, and alerts that catch drift first.

Self-service in that order: definitions, then tables, then a thin BI layer, then training. The reverse order produces dashboards that disagree.

4. What I would not do, and why scope discipline matters

- **No modelling everything.** Only the domains with a question behind them, not every landing table. A full warehouse build makes sense with a team and a multi-year horizon; alone, reporting to a CTO who needs results, it means six months without a visible number.
- **No macros or frameworks before the third repetition.** The shared incremental framework I built in my current role came after seeing the same pattern ten times. With 5 to 10 models, a macro is overhead.
- **No dashboards for everyone before definitions are stable.** My first North Star in the case sat flat at 0.92 while the thumbs-down rate went from 25% to 35%; a dashboard built on it would have been wrong for weeks.
- **No SLAs without a baseline.** A promise without a measurement is a promise you break.
- **No one-off answers that die in a Slack thread.** Every ad hoc question gets answered, and the answer becomes a model, a documented query, or a line in the definitions page. If none of the three fit, it was not worth answering.

Why it matters here: one person, four consumer teams, a product that ships continuously. In 90 days credibility comes from a few numbers people stopped arguing about, not from a well-structured repo nobody reads.

5. The biggest risks

1. **Definitions diverge before they have a home.** With 15+ modules and three teams computing their own numbers, "active user" may already mean three things. Mitigation: one layer holding definitions, and the definitions page as the place disagreements get settled.
2. **Sensitive content in the data itself.** AMA prompts are lawyers' questions. Before any table is built, agree with the CTO what the analytics layer may hold (hashed workspace and user IDs, no prompt text), who can query raw events, and where the data lives. In legal-tech, one analyst with the wrong access is a company risk, not a data risk.
3. **The North Star moves for reasons nobody logged.** A model or prompt change shifts the thumbs-down rate as much as a real quality problem, only 27% of answers are rated, and in the case data one account produced a quarter of all thumbs down. Mitigation: model and prompt version on every event as the first field in the contract, feedback rate watched next to the metric, and the per-account table read by Leadership too: five accounts are the health of the company.
4. **Priorities set only by the reporting line.** If the backlog comes from one place, I build infrastructure nobody uses. Mitigation: the weekly slot with each team, and the "which number do you not trust" list as the shared backlog that the CTO and I review together.
5. **Over-building, my own bias.** I come from a large, well-staffed build. The mitigation is the list in section 4, and a CTO who holds me to it.

Day 90, if this works: three domains with named owners, five or six tables, tests on every pull request, one definitions page people link to in arguments, and a backlog written by the teams rather than by me.